Conjugate Gradient for Linear Inverse Problems with Application to Multi-Coil MRI Reconstruction

Lecture Notes

Undergraduate Lecture Series

Department of Electrical Engineering & Computer Science

April 28, 2026

Outline

1. Motivation
2. Linear Systems as Optimization
3. Steepest Descent
4. Conjugate Gradient
5. Convergence Properties
6. Multi-Coil MRI Reconstruction
7. Summary

Why Do We Need Iterative Solvers?

Linear systems are everywhere:

- Medical imaging (CT, MRI, ultrasound)
- Weather prediction and climate modeling
- Structural mechanics (FEM)
- Machine learning (kernel methods, Gaussian processes)
- PageRank, circuit simulation, ...

Why Do We Need Iterative Solvers?

Linear systems are everywhere:

- Medical imaging (CT, MRI, ultrasound)
- Weather prediction and climate modeling
- Structural mechanics (FEM)
- Machine learning (kernel methods, Gaussian processes)
- PageRank, circuit simulation, ...

The scale problem:

- A typical MRI reconstruction: $N = 256 \times 256 = 65{,}536$ unknowns
- A 3D problem: $N = 256^3 \approx 1.7 \times 10^7$ unknowns
- Gaussian elimination costs $O(N^3)$ — completely infeasible

Why Do We Need Iterative Solvers?

Linear systems are everywhere:

- Medical imaging (CT, MRI, ultrasound)
- Weather prediction and climate modeling
- Structural mechanics (FEM)
- Machine learning (kernel methods, Gaussian processes)
- PageRank, circuit simulation, ...

The scale problem:

- A typical MRI reconstruction: $N = 256 \times 256 = 65{,}536$ unknowns
- A 3D problem: $N = 256^3 \approx 1.7 \times 10^7$ unknowns
- Gaussian elimination costs $O(N^3)$ — completely infeasible

Key Insight

Iterative methods build successively better approximations. We can stop early once the answer is *good enough*.

Two Kinds of Linear Systems

Forward Problems

Given A, $x \longrightarrow$ compute $b = Ax$

$$x \longrightarrow \boxed{A} \longrightarrow b$$

Example: Simulating MRI signal from a known image.

Inverse Problems

Given b, $A \longrightarrow$ recover x

$$x? \dashleftarrow \boxed{A^\dagger?} \longleftarrow b$$

Example: Reconstructing an image from MRI measurements.

Two Kinds of Linear Systems

Forward Problems

Given A, $x \longrightarrow$ compute $b = Ax$

$$x \longrightarrow \boxed{A} \longrightarrow b$$

Example: Simulating MRI signal from a known image.

Inverse Problems

Given b, $A \longrightarrow$ recover x

$$x? \dashleftarrow \boxed{A^\dagger?} \longleftarrow b$$

Example: Reconstructing an image from MRI measurements.

Inverse problems are harder

- Often ill-conditioned (small noise $\rightarrow$ large errors)
- May be underdetermined (more unknowns than measurements)
- A may not be square — we need a pseudoinverse, not A^{-1}
- A may be too large to store explicitly

From Linear Systems to Optimization

Setup: $A \in \mathbb{R}^{n \times n}$ symmetric positive definite (SPD), $b \in \mathbb{R}^n$.

Instead of solving $Ax = b$ directly, consider the **quadratic function**:

$$f(x) = \frac{1}{2} x^T A x - b^T x$$

From Linear Systems to Optimization

Setup: $A \in \mathbb{R}^{n \times n}$ symmetric positive definite (SPD), $b \in \mathbb{R}^n$.

Instead of solving $Ax = b$ directly, consider the **quadratic function**:

$$f(x) = \frac{1}{2} x^T A x - b^T x$$

Why? Because the gradient is:

$$\nabla f(x) = Ax - b = -(b - Ax) = -r(x)$$

where $r(x) = b - Ax$ is the *residual*.

From Linear Systems to Optimization

Setup: $A \in \mathbb{R}^{n \times n}$ symmetric positive definite (SPD), $b \in \mathbb{R}^n$.

Instead of solving $Ax = b$ directly, consider the **quadratic function**:

$$f(x) = \frac{1}{2} x^T A x - b^T x$$

Why? Because the gradient is:

$$\nabla f(x) = Ax - b = -(b - Ax) = -r(x)$$

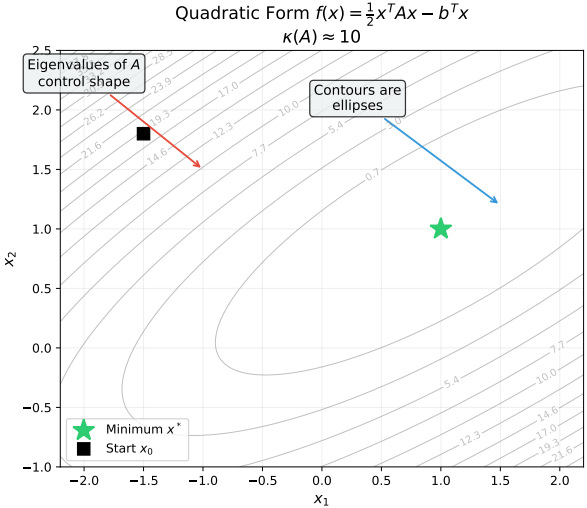
where $r(x) = b - Ax$ is the *residual*.

Equivalence

Setting $\nabla f(x^*) = 0$ gives $Ax^* = b$.
$\Rightarrow$ The unique minimizer of $f(x)$ is exactly the solution of $Ax = b$.

Geometric Intuition

In 2D, $f(x) = \frac{1}{2}x^T Ax - b^T x$ is a **paraboloid** (bowl).



Least Squares and the Normal Equations

The overdetermined case: $A \in \mathbb{R}^{m \times n}$, $m > n$

$$\min_x \ \|Ax - b\|_2^2$$

Least Squares and the Normal Equations

The overdetermined case: $A \in \mathbb{R}^{m \times n}$, $m > n$

$$\min_x \ \|Ax - b\|_2^2$$

Expand the squared norm step by step:

$$\|Ax - b\|_2^2 = (Ax - b)^T (Ax - b)$$

Least Squares and the Normal Equations

The overdetermined case: $A \in \mathbb{R}^{m \times n}$, $m > n$

$$\min_x \ \|Ax - b\|_2^2$$

Expand the squared norm step by step:

$$\begin{aligned} \|Ax - b\|_2^2 &= (Ax - b)^T (Ax - b) \\ &= x^T A^T A x - (Ax)^T b - b^T (Ax) + b^T b \end{aligned}$$

Least Squares and the Normal Equations

The overdetermined case: $A \in \mathbb{R}^{m \times n}$, $m > n$

$$\min_x \ \|Ax - b\|_2^2$$

Expand the squared norm step by step:

$$\begin{aligned}
\|Ax - b\|_2^2 &= (Ax - b)^T (Ax - b) \\
&= x^T A^T Ax - (Ax)^T b - b^T (Ax) + b^T b \\
&= x^T A^T Ax - 2b^T Ax + b^T b
\end{aligned}$$

Least Squares and the Normal Equations

The overdetermined case: $A \in \mathbb{R}^{m \times n}$, $m > n$

$$\min_x \ \|Ax - b\|_2^2$$

Expand the squared norm step by step:

$$\begin{aligned} \|Ax - b\|_2^2 &= (Ax - b)^T (Ax - b) \\ &= x^T A^T A x - (Ax)^T b - b^T (Ax) + b^T b \\ &= x^T A^T A x - 2b^T A x + b^T b \end{aligned}$$

Take the gradient and set to zero $\Rightarrow$ **normal equations**:

$$A^T A x = A^T b$$

Least Squares and the Normal Equations

The overdetermined case: $A \in \mathbb{R}^{m \times n}$, $m > n$

$$\min_x \|Ax - b\|_2^2$$

Expand the squared norm step by step:

$$\begin{aligned} \|Ax - b\|_2^2 &= (Ax - b)^T (Ax - b) \\ &= x^T A^T A x - (Ax)^T b - b^T (Ax) + b^T b \\ &= x^T A^T A x - 2b^T A x + b^T b \end{aligned}$$

Take the gradient and set to zero $\Rightarrow$ **normal equations**:

$$A^T A x = A^T b$$

Key property

$A^T A$ is always symmetric positive *semidefinite*. If A has full column rank, $A^T A$ is SPD — we can use CG!

Least Squares and the Normal Equations

The overdetermined case: $A \in \mathbb{R}^{m \times n}$, $m > n$

$$\min_x \|Ax - b\|_2^2$$

Expand the squared norm step by step:

$$\begin{aligned} \|Ax - b\|_2^2 &= (Ax - b)^T (Ax - b) \\ &= x^T A^T A x - (Ax)^T b - b^T (Ax) + b^T b \\ &= x^T A^T A x - 2b^T A x + b^T b \end{aligned}$$

Take the gradient and set to zero $\Rightarrow$ **normal equations**:

$$A^T A x = A^T b$$

Key property

$A^T A$ is always symmetric positive *semidefinite*. If A has full column rank, $A^T A$ is SPD — we can use CG!

Steepest Descent: The Natural First Attempt

Idea: At each step, move in the direction where f decreases fastest.

Algorithm 1 Steepest Descent for $Ax = b$

```
Choose initial guess x_0
for k = 0, 1, 2, ... do
    r_k = b − A x_k                          ▷ negative gradient
    α_k = (r_k^T r_k) / (r_k^T A r_k)        ▷ exact line search
    x_{k+1} = x_k + α_k r_k
end for
```

Steepest Descent: The Natural First Attempt

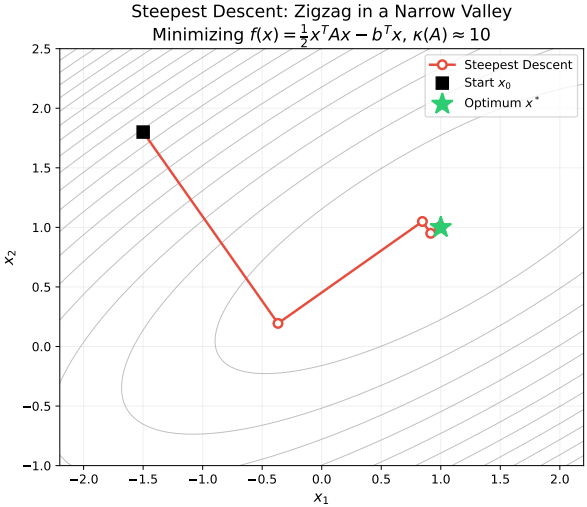
Idea: At each step, move in the direction where f decreases fastest.

Algorithm 2 Steepest Descent for $Ax = b$

```
Choose initial guess x_0
for k = 0, 1, 2, ... do
    r_k = b − A x_k                          ▷ negative gradient
    α_k = (r_k^T r_k) / (r_k^T A r_k)        ▷ exact line search
    x_{k+1} = x_k + α_k r_k
end for
```

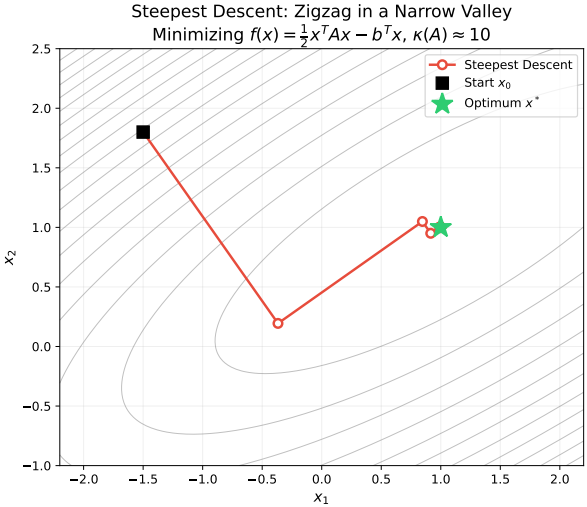
- **Line search:** α_k minimizes $f(x_k + \alpha r_k)$ exactly
- Each step is orthogonal to the *previous* step: $r_{k+1}^T r_k = 0$
- Simple, intuitive, ... but often painfully slow

Why Steepest Descent is Slow



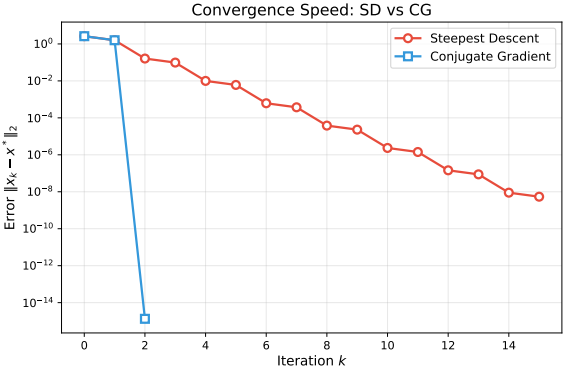
- Steepest descent **zigzags** along the narrow valley
- Each step undoes some progress of the previous step

Why Steepest Descent is Slow



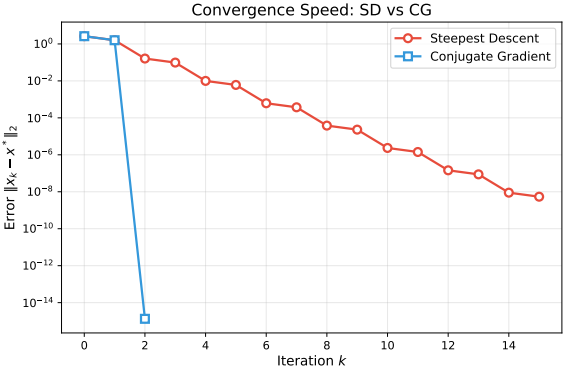
- Steepest descent **zigzags** along the narrow valley
- Each step undoes some progress of the previous step

Convergence Speed: SD vs CG



The fundamental issue: Steepest descent uses the *gradient* as search direction, but gradient directions are not independent — they interfere with each other.

Convergence Speed: SD vs CG



The fundamental issue: Steepest descent uses the *gradient* as search direction, but gradient directions are not independent — they interfere with each other.

What if we chose search directions that *don't* interfere?

This is exactly the idea behind the **Conjugate Gradient** method.

A-Orthogonal (Conjugate) Directions

Definition: Two vectors $u, v \neq 0$ are A-**orthogonal** (or *conjugate*) if:

$$u^T A v = 0$$

Geometric meaning: Under the A-inner product $\langle u, v \rangle_A = u^T A v$, conjugate directions are orthogonal.

A-Orthogonal (Conjugate) Directions

Definition: Two vectors $u, v \neq 0$ are A-**orthogonal** (or *conjugate*) if:

$$u^T A v = 0$$

Geometric meaning: Under the A-inner product $\langle u, v \rangle_A = u^T A v$, conjugate directions are orthogonal.

Why this matters:

- If we have n mutually A-orthogonal directions $\{p_0, \ldots, p_{n-1}\}$, we can reach x^* in *exactly n steps*
- Each step along p_k eliminates the error component in that direction
- Future steps don't reintroduce error in previous directions — no zigzag!

A-Orthogonal (Conjugate) Directions

Definition: Two vectors $u, v \neq 0$ are A-**orthogonal** (or *conjugate*) if:

$$u^T A v = 0$$

Geometric meaning: Under the A-inner product $\langle u, v \rangle_A = u^T A v$, conjugate directions are orthogonal.

Why this matters:

- If we have n mutually A-orthogonal directions $\{p_0, \ldots, p_{n-1}\}$, we can reach x^* in *exactly n steps*
- Each step along p_k eliminates the error component in that direction
- Future steps don't reintroduce error in previous directions — no zigzag!

But how do we find these directions efficiently?
We don't want to compute all n directions upfront ($O(n^3)$) — that defeats the purpose!

Building Conjugate Directions on the Fly

CG's key idea: Generate conjugate directions sequentially, one per iteration, using only the current residual.

The recurrence:

$$p_0 = r_0 = b - Ax_0$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

where $\beta_k = \dfrac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$ ensures $p_{k+1}^T A p_k = 0$.

Building Conjugate Directions on the Fly

CG's key idea: Generate conjugate directions sequentially, one per iteration, using only the current residual.

The recurrence:

$$p_0 = r_0 = b - Ax_0$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

where $\beta_k = \dfrac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$ ensures $p_{k+1}^T A p_k = 0$.

Properties of CG (what makes it beautiful):

1. The residuals $r_0, r_1, \ldots$ are mutually orthogonal: $r_i^T r_j = 0$ for $i \neq j$
2. The search directions are mutually A-orthogonal: $p_i^T A p_j = 0$ for $i \neq j$
3. x_k minimizes $f(x)$ over the *Krylov subspace* $\mathcal{K}_k = \text{span}\{b, Ab, \ldots, A^{k-1}b\}$
4. Only one matrix-vector product Ap_k per iteration

CG Algorithm

Algorithm 3 Conjugate Gradient (Hestenes & Stiefel, 1952)

$x_0 = 0$ (or any initial guess)
$r_0 = b - Ax_0$
$p_0 = r_0$
for $k = 0, 1, 2, \ldots$ until convergence **do**
    $\alpha_k = \dfrac{r_k^T r_k}{p_k^T A p_k}$ ▷ step size
    $x_{k+1} = x_k + \alpha_k p_k$ ▷ update solution
    $r_{k+1} = r_k - \alpha_k A p_k$ ▷ update residual
    $\beta_k = \dfrac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$ ▷ conjugacy coeff.
    $p_{k+1} = r_{k+1} + \beta_k p_k$ ▷ new direction
end for

Why CG Works: Krylov Subspace Interpretation

Krylov subspace of order k:

$$\mathcal{K}_k(A, b) = \text{span}\{b, Ab, A^2 b, \ldots, A^{k-1} b\}$$

CG chooses x_k to be the *optimal* solution within $\mathcal{K}_k$:

$$x_k = \arg \min_{x \in x_0 + \mathcal{K}_k} \|x - x^*\|_A$$

where $\|v\|_A = \sqrt{v^T A v}$ is the A-*norm* (energy norm).

Why CG Works: Krylov Subspace Interpretation

Krylov subspace of order k:

$$\mathcal{K}_k(A, b) = \text{span}\{b, Ab, A^2b, \ldots, A^{k-1}b\}$$

CG chooses x_k to be the *optimal* solution within $\mathcal{K}_k$:

$$x_k = \arg\min_{x \in x_0 + \mathcal{K}_k} \|x - x^*\|_A$$

where $\|v\|_A = \sqrt{v^T A v}$ is the A-*norm* (energy norm).

Three equivalent views of CG (they all describe the same thing):

1. An algorithm that generates A-orthogonal search directions
2. An algorithm that minimizes $\|x_k - x^*\|_A$ over $\mathcal{K}_k$
3. An algorithm that orthogonalizes the residuals via Lanczos

Finite termination

In exact arithmetic, CG finds x^* in at most n iterations. In practice, we stop much earlier — the approximation is already excellent.

Convergence Rate: The Condition Number Story

Steepest Descent (in A-norm):

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k$$

Conjugate Gradient (in A-norm):

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq 2\left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^k$$

where $\kappa = \kappa(A) = \lambda_{\max}/\lambda_{\min}$.

Convergence Rate: The Condition Number Story

Steepest Descent (in A-norm):

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k$$

Conjugate Gradient (in A-norm):

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq 2\left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^k$$

where $\kappa = \kappa(A) = \lambda_{\max}/\lambda_{\min}$.

CG replaces κ with $\sqrt{\kappa}$ — a dramatic improvement!

κ	SD factor/iter	CG factor/iter
1	0.000 (instant)	0.000 (instant)
10	0.818	0.520
100	0.980	0.819
1000	0.998	0.939

Convergence Rate: The Condition Number Story

Steepest Descent (in A-norm):

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k$$

Conjugate Gradient (in A-norm):

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq 2\left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^k$$

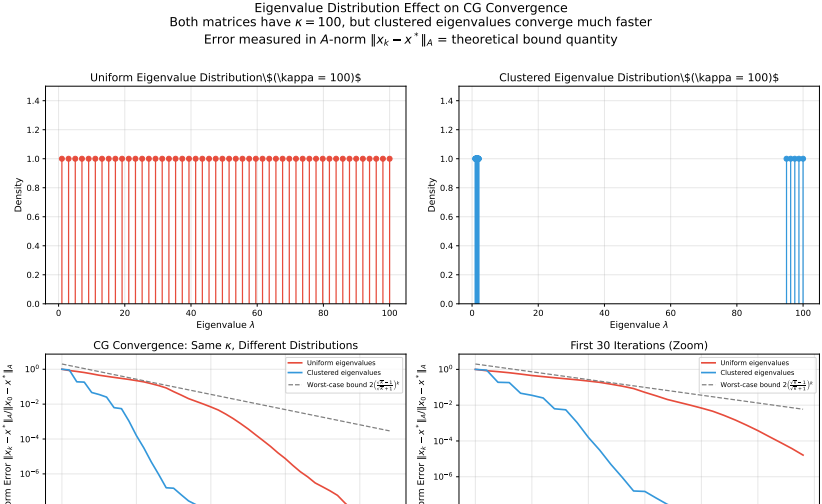
where $\kappa = \kappa(A) = \lambda_{\max}/\lambda_{\min}$.

CG replaces κ with $\sqrt{\kappa}$ — a dramatic improvement!

κ	SD factor/iter	CG factor/iter
1	0.000 (instant)	0.000 (instant)
10	0.818	0.520
100	0.980	0.819
1000	0.998	0.939

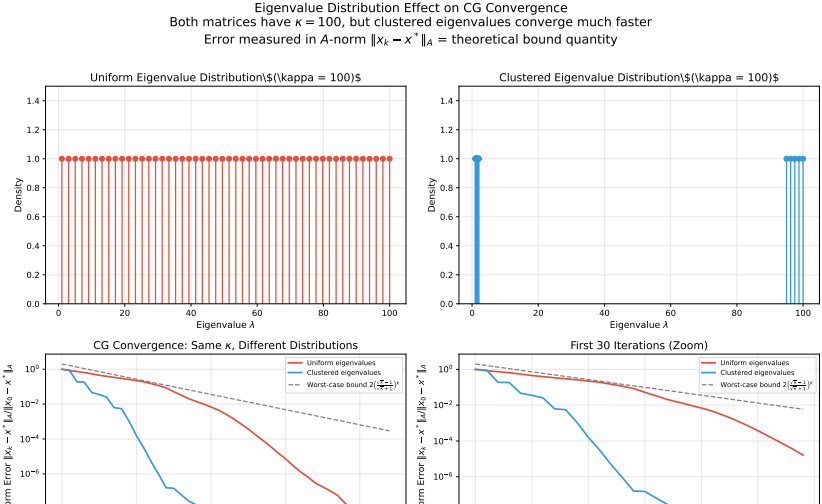
Beyond the Condition Number: Eigenvalue Clustering

The bound in terms of κ is a *worst-case* bound. Real convergence depends on the **distribution** of eigenvalues. **CG typically converges much faster than the bound predicts!**



Beyond the Condition Number: Eigenvalue Clustering

The bound in terms of κ is a *worst-case* bound. Real convergence depends on the **distribution** of eigenvalues. **CG typically converges much faster than the bound predicts!**



Practical CG: Stopping Criteria and Preconditioning

When to stop?

- Monitor $\|r_k\|_2/\|b\|_2$ (relative residual)
- Stop when below a tolerance (e.g., 10^{-6} for well-conditioned systems)
- In inverse problems: stop before fitting the noise! (This is the *semiconvergence* phenomenon — the error to the true solution may increase after some point, even though the residual keeps decreasing)

Practical CG: Stopping Criteria and Preconditioning

When to stop?

- Monitor $\|r_k\|_2/\|b\|_2$ (relative residual)
- Stop when below a tolerance (e.g., 10^{-6} for well-conditioned systems)
- In inverse problems: stop before fitting the noise! (This is the *semiconvergence* phenomenon — the error to the true solution may increase after some point, even though the residual keeps decreasing)

Preconditioning strategies for CG:

- **Jacobi (diagonal):** $M = \text{diag}(A)$ — simple, sometimes effective
- **Incomplete Cholesky:** Approximate factorization with limited fill
- **Multigrid:** Excellent for elliptic PDEs, $O(n)$ complexity
- **For MRI specifically:** Density compensation, circulant preconditioners

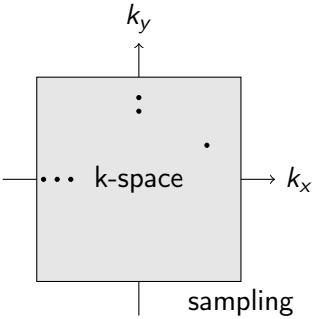
MRI in One Slide

Magnetic Resonance Imaging measures the signal from excited protons in a strong magnetic field.

- Protons precess at the Larmor frequency $\omega = \gamma B_0$
- **Gradient fields** G_x, G_y, G_z encode spatial position into frequency and phase
- The measured signal in **k-space** is the 2D Fourier transform of the proton density (image):

$$s(k_x, k_y) = \iint \rho(x, y)\, e^{-j2\pi(k_x x + k_y y)}\, dx\, dy$$

- **k-space** = spatial frequency domain of the image



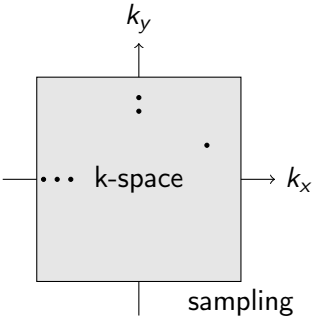
MRI in One Slide

Magnetic Resonance Imaging measures the signal from excited protons in a strong magnetic field.

- Protons precess at the Larmor frequency $\omega = \gamma B_0$
- **Gradient fields** G_x, G_y, G_z encode spatial position into frequency and phase
- The measured signal in **k-space** is the 2D Fourier transform of the proton density (image):

$$s(k_x, k_y) = \iint \rho(x, y)\, e^{-j2\pi(k_x x + k_y y)}\, dx\, dy$$

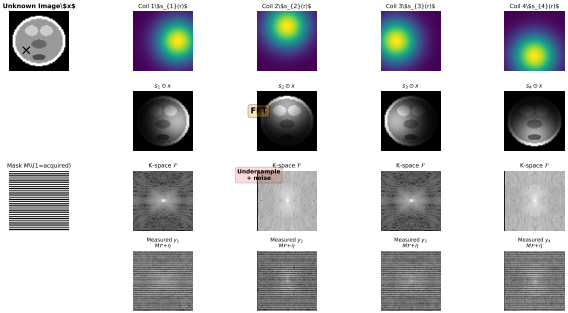
- **k-space** = spatial frequency domain of the image



Multi-Coil Acquisition: The Key to Acceleration

Modern MRI scanners use **arrays of receiver coils** placed around the patient.

Multi-Coil MRI Forward Model: $y_i = M\mathcal{F}\{(S_i \odot x)\} + \eta_i$

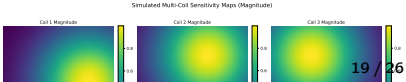


SENSE Inverse Problem

$$\min_x \sum_{i=1}^{C} \|M\mathcal{F}S_i x - y_i\|_2^2$$

Solve with Conjugate Gradient

- Each coil i has a **sensitivity profile** $s_i(r)$ — it sees the object differently depending on position



The SENSE Model (Pruessmann et al., 1999)

For C coils with sensitivities $\{s_i(\mathrm{r})\}_{i=1}^{C}$, the forward model:

$$y_i = M\mathcal{F}\{s_i \odot x\} + \eta_i, \qquad i = 1, \ldots, C$$

- $x \in \mathbb{C}^N$: unknown image (flattened)
- $s_i \in \mathbb{C}^N$: coil sensitivity map (diagonal operator S_i)
- $\mathcal{F}$: 2D discrete Fourier transform (unitary)
- M: Cartesian undersampling mask (keeps every R-th line)
- y_i: measured k-space data from coil i
- η_i: complex Gaussian noise

The SENSE Model (Pruessmann et al., 1999)

For C coils with sensitivities $\{s_i(\mathrm{r})\}_{i=1}^{C}$, the forward model:

$$y_i = M\mathcal{F}\{s_i \odot x\} + \eta_i, \qquad i = 1, \ldots, C$$

- $x \in \mathbb{C}^N$: unknown image (flattened)
- $s_i \in \mathbb{C}^N$: coil sensitivity map (diagonal operator S_i)
- $\mathcal{F}$: 2D discrete Fourier transform (unitary)
- M: Cartesian undersampling mask (keeps every R-th line)
- y_i: measured k-space data from coil i
- η_i: complex Gaussian noise

The SENSE inverse problem (least squares):

$$\min_x \sum_{i=1}^{C} \left\| M\mathcal{F}S_i x - y_i \right\|_2^2$$

This has the form $\min_x \|Ax - y\|_2^2$ — we can apply **CG**!

SENSE Normal Equations for CG

Step 1: Form the normal equations

Expanding the least-squares cost and setting the gradient to zero:

$$\underbrace{\left(\sum_{i=1}^{C} S_i^H \mathcal{F}^H M \mathcal{F} S_i\right)}_{A_{\text{norm}}} x = \underbrace{\sum_{i=1}^{C} S_i^H \mathcal{F}^H y_i}_{b}$$

Step 2: Apply A_{norm} implicitly via FFTs

For any vector v, compute $A_{\text{norm}} v$ without forming the matrix:

1. For each coil i: multiply by s_i, FFT, apply mask M, IFFT, multiply by $\overline{s_i}$
2. Sum over all coils

Step 3: Ensure positive definiteness

A_{norm} is Hermitian positive *semidefinite*. With aggressive undersampling, its nullspace may contain nonzero vectors, causing CG to break down. The fix: add a small **Tikhonov regularization** term:

$$(A_{\text{norm}} + \lambda I)\, x = b, \qquad \lambda > 0$$

SENSE Normal Equations for CG

Step 1: Form the normal equations

Expanding the least-squares cost and setting the gradient to zero:

$$\underbrace{\left(\sum_{i=1}^{C} S_i^H \mathcal{F}^H M \mathcal{F} S_i\right)}_{A_{\text{norm}}} x = \underbrace{\sum_{i=1}^{C} S_i^H \mathcal{F}^H y_i}_{b}$$

Step 2: Apply A_{norm} implicitly via FFTs

For any vector v, compute $A_{\text{norm}} v$ without forming the matrix:

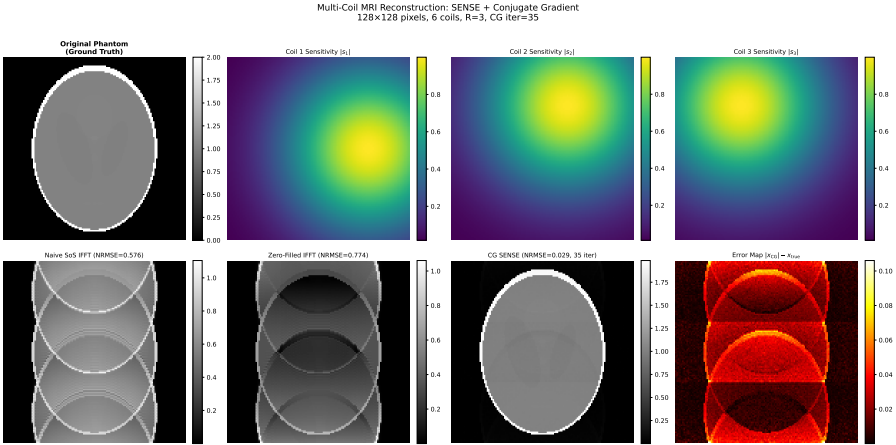
1. For each coil i: multiply by s_i, FFT, apply mask M, IFFT, multiply by $\overline{s_i}$
2. Sum over all coils

Step 3: Ensure positive definiteness

A_{norm} is Hermitian positive *semidefinite*. With aggressive undersampling, its nullspace may contain nonzero vectors, causing CG to break down. The fix: add a small **Tikhonov regularization** term:

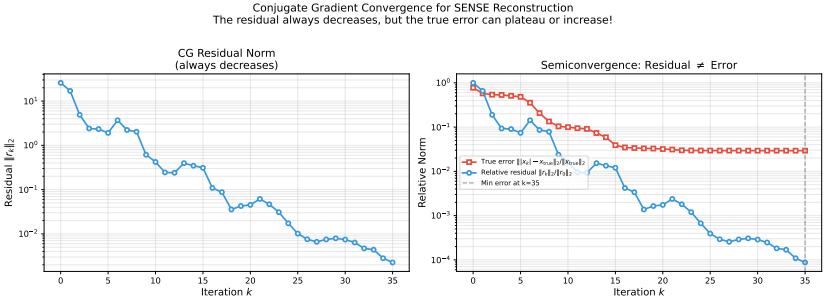
$$(A_{\text{norm}} + \lambda I)\, x = b, \qquad \lambda > 0$$

Reconstruction Results: Zero-Filled vs CG



- **Zero-filled IFFT:** Fast but shows severe aliasing artifacts (overlapping replicas from undersampling)
- **CG (SENSE):** Uses coil sensitivity information to separate the overlapping signals — aliasing is removed

CG Convergence and Semiconvergence



- Residual drops rapidly in the first few iterations
- After ∼15 iterations, we're close to the true image
- **Warning:** In inverse problems, the residual *always* decreases monotonically, but the true reconstruction error can start to *increase* as we begin to fit the noise — this is **semiconvergence**
- The effective condition number depends on:
  - Coil geometry (well-separated coils → better conditioning)
  - Undersampling factor R (higher R → worse conditioning)

Summary: What We Learned

1. **Iterative solvers** are essential for large-scale linear systems where direct methods are too expensive.
2. **Steepest descent** is intuitive but suffers from zigzagging — convergence depends on $\kappa(A)$, with rate $((\kappa-1)/(\kappa+1))^k$.
3. **Conjugate Gradient** fixes this by using A-orthogonal search directions, replacing κ with $\sqrt{\kappa}$ in the convergence bound, and converging in at most n steps.
4. **Convergence** depends on eigenvalue distribution, not just condition number — clustered eigenvalues mean faster convergence. The condition number bound is a worst-case; real CG is often much faster.
5. **Multi-coil MRI** uses arrays of receiver coils with distinct sensitivity profiles to enable undersampled acquisition.
6. **SENSE reconstruction** formulates the problem as a linear least-squares system; CG solves it efficiently using implicit matrix-vector products via FFTs. Regularization ($+\lambda I$) ensures positive definiteness and controls noise amplification.

CG is everywhere

Further Reading and Code

Classic papers:

- Hestenes & Stiefel (1952). "Methods of Conjugate Gradients for Solving Linear Systems." *J. Res. Natl. Bur. Stand.*
- Pruessmann et al. (1999). "SENSE: Sensitivity Encoding for Fast MRI." *Magn. Reson. Med.*

Textbooks:

- Trefethen & Bau (1997). *Numerical Linear Algebra*. SIAM.
- Nocedal & Wright (2006). *Numerical Optimization*. Springer.
- Shewchuk (1994). "An Introduction to the Conjugate Gradient Method Without the Agonizing Pain." (free online — highly recommended!)

Code for this lecture:

- `code/illustrations/` — Python scripts that generate the figures
- `code/mri_demo/` — Full SENSE+CG reconstruction demo
- All code is self-contained and runnable; experiment with different parameters to build intuition!

Thank you!

Questions?

"The CG method is the most important iterative method for solving large sparse systems of linear equations."
— Yousef Saad